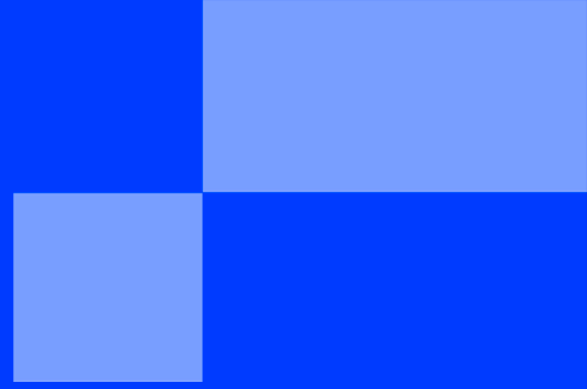




SUMMER SCHOOL

< PART 1 - HTML & CSS FUNDAMENTALS />

SUMMER SCHOOL

< WEB DEVELOPMENT: BUILDING THE NEWSPAPER />

"First, solve the problem. Then, write the code."
— **John Johnson**

The Case

Welcome to the technical team! The Editor-in-Chief of the new School Digital Newspaper has officially requested the very first visual prototype of the media portal. Currently, the editorial team has great content but nowhere to publish it. They need a digital presence that is accessible, readable, and structured.

Your mission is to build the digital foundation of the newspaper from scratch. You will need to create a clean, maintainable, and presentable front-end foundation that structures the news logically and styles it beautifully.

Ground Rules: The "Vanilla" Constraint

This module focuses strictly on Front-End Fundamentals.



Strict Restriction: You are **NOT** allowed to use any CSS or JavaScript frameworks for this project. No Tailwind, no Bootstrap, no React. Your work must be entirely built using **vanilla HTML5 and CSS3**. The goal is to master the core building blocks of the web before using tools that generate them for you.

Theory: The Building Blocks

Before you begin building the prototype, you need to understand the fundamental mechanics of the web.

1. HTML: The Skeleton and Semantics

HTML (HyperText Markup Language) describes the structure of your page using **tags**. A tag acts as a container for your text, images, or even other tags.

In HTML, the most basic, generic container is called a `<div>` (for division). It creates an invisible box on your page. However, if you build an entire website using only `<div>` tags, the web browser and search engines will have no idea what those boxes represent.

To fix this, modern web development uses **semantic tags**. These tags behave exactly like a generic `<div>` box, but they tell the browser exactly what the content means. You must use them to structure your newspaper:

- `<header>`: For your top navigation area and website logo.
- `<main>`: To wrap the core content of your page.
- `<section>` and `<article>`: To group related news blocks.
- `<footer>`: For the bottom of your page.

2. CSS: The Paint and Layout

If HTML is the skeleton, **CSS (Cascading Style Sheets)** is the skin. To apply styles, your HTML file needs to know where your CSS file is located. Add this line inside the `<head>` of your `index.html`:

```
<link rel="stylesheet" href="styles.css">
```

CSS works by targeting an HTML element and applying rules to it. Remember that every element on a web page is a rectangular box. You manage the space inside the box with `padding`, and the space outside the box with `margin`.

To align elements side-by-side (like a navigation menu or an article grid), you should use a CSS layout module called **Flexbox**:

```
.article-grid {  
  display: flex; /* Turns the container into a flexible row */  
  justify-content: space-between; /* Spreads the items out */  
  gap: 20px; /* Adds space between the items */  
}
```

Project Requirements

This project will not hold your hand step-by-step. You are expected to analyze the requirements, set up your files, and implement a clean render independently.

1. Minimal Architecture

You must deliver the following simplified file structure:

- `index.html` (The newspaper's homepage)
- `styles.css` (Your dedicated stylesheet)
- A short `README.md` containing a brief explanation of your project and instructions on how to open it.

2. The Homepage Structure (`index.html`)

The main landing page must act as a functional news portal. It must include:

- ✓ **A Header:** Containing the newspaper's branding (name/logo) and basic dummy navigation links (e.g., Home, Politics, Campus Life).
- ✓ **A Hero Section (Main body of the page):** Highlighting the main breaking news article.
- ✓ **An Article Grid:** Displaying multiple smaller news articles. Each `<article>` card must feature a title, a short text excerpt, and a "Read More" button.
- ✓ **A Footer:** Containing dummy copyright information and social media links.

3. CSS & Readability (`styles.css`)

Apply CSS rules to organize your content logically. You are not evaluated on flashy designs, but rather on UX (User Experience) fundamentals:

- **Typography:** Ensure high readability. Choose clean sans-serif or serif fonts and ensure your line heights are comfortable for reading long texts.
- **Spacing:** Use consistent margins and padding to let the layout breathe. Do not cram text against the edges of the screen.
- **Contrast:** Ensure text stands out clearly against background colors.



Think like a product designer: A simple, solid, and highly readable interface is infinitely better than a heavily "styled" but broken layout.

v1

{EPITECH}